Nanowork Pipeline Debugging & Refactor — Session Summary

A working reference for the engineering arc covered in this session. Organized chronologically so you can trace how each issue was diagnosed and fixed, with the key decisions and tradeoffs preserved.

Architecture (as of session start)

The original pipeline:

1. User chats with the agent on Linq → describes their business
2. Agent extracts context, runs `_classify_build_type` to pick `tool_app` vs `business`
3. Agent generates a React-based HTML page (in-browser Babel + eval pattern)
4. Pre-payment: deploy to Vercel via GitHub repo (create repo → push files → Vercel pulls)
5. Post-payment: full multi-page SDLC pipeline (`gather_requirements → design_architecture → build_pages → run_tests → deploy`)
6. User receives URL via Linq message

Stack:

- Render hosts the FastAPI agent service
- Supabase for `linq_waitlist` and `pitch_pages` tables
- Anthropic streaming for code generation
- Gemini for hero/logo image generation
- Vercel for hosting generated pages
- GitHub for post-payment repo creation

Issue 1 — "Only 3 experiences ever appear: 404, blank page, prebuilt template"

Diagnosis

The pipeline was always emitting one of three Vercel fallback states regardless of input. Logs showed:

- `POST /user/repos` → 403 Forbidden
- Despite GitHub failure, code continued to call Vercel
- Vercel deployment succeeded with no real source → fallback page served
- User got "your site is live" message pointing to broken URL

Root cause: GitHub repo creation was failing silently and the pipeline kept going to Vercel anyway. The 403 was a token permission issue (fine-grained PAT missing `Administration: write` or classic PAT missing `repo`).

Decision: scrap GitHub for pre-payment, keep it for post-payment

Tradeoff analysis:

- Pre-payment is one-shot static HTML, no editing, no user ownership → GitHub adds failure surface for no benefit
- Post-payment is real multi-page projects where GitHub actually helps (build pipeline, history, eventual user export)

Implementation (Composer prompt: `composer2-deploy-refactor-prompt.md`)

- Created `vercel_direct_deploy.py` with `deploy_static_bundle` for inline file upload to Vercel API
- Pre-payment now uses direct Vercel upload, no GitHub at all
- Post-payment retains GitHub flow with `strict=True` flag — fails loud if GitHub fails, never proceeds to Vercel
- Added structured `DEPLOY[*]` logging at every decision point
- Added `GitHubRepoCreateError` typed exception with retry-on-rate-limit
- Added `verify_github_token` precheck that runs at startup

Outcome

Pre-payment deploy refactor shipped successfully. Tests passed. Architecture sound.

Issue 2 — Pipeline hangs mid-LLM-stream

Diagnosis

After the deploy refactor, generations would start but never reach `DEPLOY[start]`. Trace would end mid-stream during `generate_landing_page`.

Identified causes:

1. No watchdog timeout on the streaming Anthropic call
2. No httpx-level read timeout (default infinite)
3. No completion log line, so we couldn't tell if the stream finished or hung
4. Render's `==> Detected service running on port 10000` mid-build looked like a restart but was actually status reminder

Recommended fix (not yet implemented)

- Add `asyncio.wait_for(consume_stream(...), timeout=300)` on streaming calls
- Set `httpx.Timeout(connect=10.0, read=60.0, ...)` on the Anthropic client
- Log `LLM stream END` at completion mirroring the existing `LLM stream START`
- Add instance-ID logging to distinguish multi-instance vs. single-instance hangs

Status: flagged for follow-up. Not blocking subsequent issues.

Issue 3 — Blank page bug (after deploy refactor shipped)

Diagnosis arc

This took multiple rounds because each fix revealed the next bug:

Round 1: "Page is blank, View Source shows my HTML" → narrowed to: HTML reaches Vercel, browser receives it, but React doesn't mount.

Round 2: Console diagnostics showed `typeof App === 'undefined'` after eval. Source contained `function App()` and was 35,994 chars. Source was NOT truncated.

Round 3: The smoking gun — `compiled.startsWith('"use strict"')` was `true`. Babel's `env` preset was injecting `"use strict"`, and **in strict mode, function declarations inside `eval()` do NOT leak to the surrounding scope.** So the eval ran successfully, `App` got declared inside the eval scope, then disappeared when eval returned.

Implementation (Composer prompt: `composer2-strict-mode-eval-fix-prompt.md`)

Two-layer fix:

1. **Hoist to `window`:** Modified the bootstrap to inject `window.App = App` (and helpers) immediately after compiled code runs, then reference `window.App` for the existence check and the `ReactDOM.render(...)` call.
2. **Drop the `env` preset:** Changed `Babel.transform(src, { presets: ['env', 'react'] })` to just `presets: ['react']`. Modern browsers don't need ES5 transpilation, and removing `env` removes the `"use strict"` injection.

Outcome

Blank page fixed. Now seeing a different error overlay — Issue 4.

Issue 4 — Compile error from agent-generated JSX

Diagnosis

After the strict-mode fix, the error overlay correctly fired with:

```
Compile error: Unexpected token (652:2)
  650 |     </div>
  651 |   );
> 652 |
```

Root cause: agent's JSX was missing the closing `}` for `function App()`. Babel parsed everything until EOF, never found the closing brace, threw "Unexpected token" at trailing whitespace.

But further investigation revealed a deeper issue — the `syntax_agent` had a "safety net" that was *creating* this kind of bug.

Issue 5 — `syntax_agent` regex brace counter was making things worse

The deepest diagnosis of the session

Production log showed:

```
WARNING syntax_agent: Pass 1 pre-check found 1 issue(s):
        Unmatched braces: net open count = -1
        (positive = missing closes, negative = extra closes)

WARNING syntax_agent: balance_braces: SAFETY NET FIRED —
        removed extra `}` (removal 1/1) near:
        ' <Footer setCurrentPage={setCurrentPage} />\n    </div>\n  );\n}'

INFO    syntax_agent: Pass 1 clean — skipping Pass 2
```

What actually happened:

1. LLM produced valid JSX. The `net = -1` count was a **false positive** — the regex counter doesn't understand JSX expression containers (`{expr}`), template literals, or escaped braces in strings
2. The safety net removed the trailing `}` of `function App()` — the most important brace in the entire output
3. The "Pass 1 clean" log lied — Pass 1 mutated the source

The safety net was producing bugs while logging "clean." Same antipattern that would later show up in `assembly_agent`.

Implementation (Composer prompt: `composer2-syntax-agent-fix-prompt.md`)

- Created `jsx_validator.py` that shells out to Node + `@babel/parser` for real validation
- Created `validate_or_regenerate` flow: validate → if invalid, ask LLM to regenerate with parser error included → max 2 attempts → raise `SyntaxAgentValidationError`
- **Deleted** `balance_braces` and the regex counter entirely
- Replaced misleading "Pass 1 clean" logs with structured logs reflecting actual state

Sub-issue: missing npm dependency on Render

After deploying, generations failed with `MODULE_NOT_FOUND: '@babel/parser'`. The Node script was correct but the dependency wasn't installed on Render.

Fix: Updated Render's Build Command from `uv sync` to `npm install --omit=dev && uv sync`. Required adding `package.json` to repo root with `@babel/parser` as a dependency.

Outcome

Syntax agent now validates with real Babel parser. Broken JSX fails loud instead of being silently mutated. Pre-payment deploys work end-to-end.

Issue 6 — Vercel SSO protection blocking public access

Diagnosis

Generated pages deployed successfully but required Vercel team SSO login to view. URL returned 401 in incognito.

Root cause: Vercel team had Deployment Protection enabled by default; new projects inherited the SSO lock.

Decision: code fix, not dashboard fix

Even though the dashboard fix was simpler, the user explicitly chose the code path so the fix is enforced regardless of dashboard config drift.

Implementation (Composer prompts: `composer2-vercel-public-access-prompt.md` then `composer2-vercel-protection-fix-correction-prompt.md`)

First attempt failed: Composer added `ssoProtection: null` to both the deployment POST body and a follow-up project PATCH. Vercel rejected the deployment POST with:

```
"should NOT have additional property `ssoProtection`. Please remove it."
```

Correction: `ssoProtection` and `passwordProtection` are project-level settings, not deployment-level. The corrected fix:

- Removed both fields from `POST /v13/deployments` body
- Kept the `PATCH /v9/projects/{name}` call as best-effort (logs warning on failure, doesn't fail deploy)

Added regression test: `test_deploy_static_bundle_post_body_matches_vercel_schema` — asserts the deploy body keys match a documented allowlist. Catches "we accidentally added an unknown field" before it hits Vercel.

Issue 7 — "Always emit landing page" → build-type routing

The architectural ask

Currently every generation produces a marketing landing page even if the user describes a calculator, contact form, etc. Goal: agent should produce the right kind of app based on what the user asked for.

Decisions

- **Closed taxonomy of 8 types + `other` fallback:** `landing`, `tool`, `form`, `directory`, `portfolio`, `booking`, `app`, `info`, `other`
- **Classification at context-extraction time** (one extra field, no separate classifier agent)
- **Per-type prompt files**, no shared base (intentional duplication for editing isolation)
- **Both pre-payment and post-payment** routed through same classifier
- **Default to `landing` on low confidence**, log warning, no chat clarification step

Pre-implementation discovery (Composer report)

Composer flagged a real conflict before writing code:

`build_type` key collision: Existing codebase uses `context_json.build_type` with values `tool_app | business` for high-level pipeline routing. New taxonomy wants the same key for `landing | tool | form | ...`. Same key, different value spaces, different layers.

Resolution decided:

- Rename legacy field to `build_track` (keeps `tool_app | business` semantics for pipeline selection)
- Use `build_type` for new taxonomy (picks prompt template within a pipeline)
- They're orthogonal: a `business` track build can have `build_type: "landing"` or `"tool"` etc.
- Migration backfills existing rows: legacy `business` → `build_track="business", build_type="landing"`. Legacy `tool_app` → `build_track="tool_app", build_type="tool"`.

Post-payment scope reduction: `release_phase` and `post_payment_pipeline` don't actually call `generate_landing_page` — they use `assemble_final_site` from already-built JSX or `routers/orchestration.generate_site`. So this PR scopes to pre-payment only; post-payment plumbing is a follow-up.

Status: prompt sent (`composer2-build-type-routing-final-prompt.md`), implementation pending.

Issue 8 — Three failures in one post-payment build (most recent)

Three distinct problems in one trace

Production build for slug `labssh-42ff2359` revealed:

Failure 1: `assembly_agent` silently swallows validation errors

Same antipattern as the regex brace counter, in a different file:

```
ERROR assembly_agent: Assembly: sanity_check_and_fix failed for Labssh —
      shipping best-effort pre-QA JSX rather than failing the whole build
INFO  final_testing_agent: QA pass skipped — precheck and parse validation clean
INFO  assembly_agent: Assembly complete for Labssh
```

`assembly_agent.py` line 232 catches `SyntaxAgentValidationError` and continues with broken JSX. Then `final_testing_agent` falsely logs "validation clean" when validation actually failed. The validator infrastructure was being undermined at the layer above.

Failure 2: 167,282 chars of JSX source

Post-payment generation produced 3-5x the size of pre-payment landing pages. Likely contributor to the validation failures (more surface area = more bugs). May be due to duplicated shared components per page in the stitcher, or legitimately needed multi-page JSX.

Failure 3: GitHub 403 returned

Same token issue from Issue 1 reappearing. Token (`jordantplows`) lacks repo creation permission. This time the strict-flag pipeline correctly aborted instead of continuing — but the underlying token hasn't been fixed.

Implementation (Composer prompt: `composer2-post-payment-three-fixes-prompt.md`)

- **Part 1:** Remove the catch in `assembly_agent.py`. Let `SyntaxAgentValidationError` propagate. Fix the misleading `final_testing_agent` log. Added regression test using `ast` module to verify no `except SyntaxAgentValidationError` block exists in `assembly_agent.py` ever again.
- **Part 2:** Fix the `final_testing_agent` log to only fire when validation actually passed.

- **Part 3:** Investigate JSX bloat. Add structured `source_breakdown` logging. Either deduplicate or document that the size is intentional.
- **Part 4:** Add `GitHubTokenPermissionError` subclass that detects 403 + `"Resource not accessible by personal access token"` body. User-facing message becomes specific: "Our GitHub access token needs to be updated."
- **Part 5:** Operator checklist for the human to actually reissue the GitHub token.

Status: prompt sent, implementation pending.

Side issues observed throughout

These came up in logs and were flagged but not blocking:

- **Stripe `success_url` invalid** — every payment flow returns 400 from Stripe with `Not a valid URL`. User gets a hardcoded placeholder link instead of real Stripe checkout. **Revenue-critical, still unfixed.**
- **Linq reactions API 400** — every reaction post fails with `operation must be either 'add' or 'remove'`. Spam in logs but not blocking.
- **`_detect_user_persona` JSON parse failure** — falls back to defaults silently. LLM occasionally returns non-JSON.
- **Render's `==> Detected service running on port 10000`** appearing mid-build — initially suspected as restart, confirmed as benign status reminder (no `Shutting down` line nearby).

Architectural patterns identified

The "catch and continue with broken state" antipattern

Appeared in three places, same shape:

1. Old `syntax_agent.balance_braces` — caught "wrong" brace count, mutated source, logged "clean"
2. `assembly_agent.assemble_final_site` — caught `SyntaxAgentValidationError`, kept broken JSX, logged "complete"
3. `release_phase.deploy` (original) — GitHub 403 caught, Vercel called anyway, logged "deployed"

Each instance silently shipped broken state to users while logs claimed success. The fix in every case: **fail fast, raise typed exceptions, surface to user.**

The in-browser Babel/eval architecture's fragility

The current pattern (LLM emits JSX → browser loads CDN scripts → in-browser Babel transforms → eval → mount) has been the source of multiple distinct bug classes:

- Strict-mode scope traps `App` in eval
- LLM token-limit truncation produces partial JSX
- LLM brace-balance errors cause Babel parse failures
- CDN block (ad blockers, corporate networks) silently fails
- Tailwind CDN compilation in-browser is slow and fragile

Recommended (not yet pursued): pre-build with Vite/esbuild on the server so syntax errors are caught at build time, not view time. Smaller bundle, faster load, immune to CDN blocks. Multiple times deferred during this session because of scope.

The Composer prompt pattern

Most successful prompts had this shape:

1. **Read these files first** with explicit listing requirement
2. **Confirmed diagnosis** preventing re-investigation
3. **Numbered parts** scoped to specific files
4. **Things you must NOT do** to prevent scope creep
5. **Verification commands** with specific grep patterns
6. **What to include in your final reply** as a checklist

Less successful patterns:

- Asking for behavioral verification (curl tests) without forcing actual execution
- Conflating code fix with infrastructure config (npm install on Render)
- Assuming Vercel API field names without doc verification

Composer prompts produced this session (in order)

1. `composer2-deploy-refactor-prompt.md` — split GitHub from pre-payment, fail-fast pipeline
2. `composer2-blank-page-diagnostic-prompt.md` — instrumentation prompt for blank page debugging
3. `composer2-strict-mode-eval-fix-prompt.md` — hoist to window, drop env preset
4. `composer2-syntax-agent-fix-prompt.md` — replace regex counter with Babel parser

5. `composer2-vercel-public-access-prompt.md` — disable Vercel SSO programmatically (had bug)
6. `composer2-vercel-protection-fix-correction-prompt.md` — corrected (only PATCH, not POST)
7. `composer2-build-type-routing-final-prompt.md` — taxonomy + per-type prompts
8. `composer2-post-payment-three-fixes-prompt.md` — assembly_agent + bloat + GitHub error UX

Outstanding items (snapshot at end of session)

Implementation pending:

- Build-type routing (Composer prompt sent, awaiting completion)
- Post-payment three fixes (Composer prompt sent, awaiting completion)

Operator action required:

- Reissue GitHub token for `jordantplows` with `repo` scope (classic PAT) or `Administration: write` (fine-grained PAT)
- Update `GITHUB_TOKEN` env var on Render after reissuing
- Without this, post-payment builds will continue to fail regardless of code

Flagged for follow-up:

- Streaming watchdog timeout on Anthropic calls
- Stripe `success_url` fix (revenue-critical)
- Linq reactions API 400 (cosmetic but log noise)
- `_detect_user_persona` JSON parse robustness
- User-override mechanism for build-type misclassification
- Post-payment plumbing of `build_type` through `release_phase` and `post_payment_pipeline`
- Architectural decision on whether to move away from in-browser Babel/eval pattern

Watch in production after each shipped fix:

- `DEPLOY[done] outcome=success` rate by `flow_type` and `build_type`
- `syntax_agent: validation FINAL_FAIL` rate (should be near zero in steady state)
- `assembly_agent: validation_FAILED` rate (post-payment specific)
- `disable_protection_OK` vs `disable_protection_FAILED` rate (Vercel access)

- Per-type build success rates once routing ships (especially `app` type)

This document was generated as a working reference, not a verbatim transcript. For exact wording of any decision or prompt, refer to the original chat history.